Mission - Assistant de boîte mail : Web, API & Automatisation n8n

Contexte

Cet exercice prolonge l'exercice précédent (consultation des e-mails du jour et réponses depuis une interface web). Il vise cette fois à évaluer votre capacité à construire une application **fiable dans la durée** : synchronisation incrémentale, absence de doublons, absence de double envoi, IA encadrée et sécurité des échanges entre les briques.

Les attentes portent sur la **qualité du code**, la **pertinence de l'architecture**, la **robustesse** face aux erreurs et l'**expérience utilisateur**.

Durée conseillée : 1 semaine.

Objectifs

1. Développer une **interface web responsive** de type boîte de réception : liste, filtres, fil de discussion et réponse.
2. Développer un **backend API** avec une base de données, placé entre le site et n8n : le site ne communique jamais directement avec n8n.
3. Mettre en place des **workflows n8n** de synchronisation incrémentale, de classification par IA et d'envoi des réponses.
4. Proposer des **réponses rédigées par IA** qui restent sous le contrôle de l'utilisateur et sont envoyées dans le bon fil de discussion.

Partie 1 – Interface web

Framework libre (React, Vue, Angular, Svelte…).

Liste des e-mails

- Expéditeur, objet, date et heure de réception
- Catégorie et priorité attribuées par l'IA
- Statut de traitement : *à traiter*, *brouillon prêt*, *répondu*, *ignoré*
- Pagination (la liste n'est plus limitée à la journée)

Fonctionnalités

- Filtres **combinables** : catégorie, priorité, statut, expéditeur, période, et recherche dans l'objet et le contenu
- Vue détail affichant le **fil de discussion complet** (tous les messages du même échange, dans l'ordre)

- Bloc « **Aujourd'hui** » : résumé global de la journée et liste des 5 e-mails à traiter en priorité, avec la raison de chaque choix
- Réponse manuelle depuis la vue détail
- Réponse par IA :
  - l'IA produit un **brouillon modifiable**, avec un ton au choix (formel, neutre, court) ;
  - aucun e-mail n'est envoyé sans **validation explicite** de l'utilisateur ;
  - l'état de l'envoi est affiché : en cours, envoyé, échec avec bouton « Réessayer ».
- Accès protégé par une **connexion** : aucune donnée visible sans être authentifié
- Interface **responsive** utilisable sur desktop et mobile

Partie 2 – Backend API

Langage libre (Node.js, PHP/Symfony, Python…). Le fichier JSON de l'exercice précédent est remplacé par une **base de données** (SQLite ou PostgreSQL).

- Modélisez au minimum : e-mails, fils de discussion, brouillons, réponses envoyées, exécutions de synchronisation.
- Exposez des routes REST documentées, par exemple :
  - `GET /emails` (filtres + pagination), `GET /threads/{id}`
  - `GET /summary?date=…`
  - `POST /emails/{id}/draft` (demande de brouillon IA), `POST /emails/{id}/reply`
- Recevez les données de n8n par un **webhook sécurisé** : secret partagé transmis dans un en-tête, requête refusée sinon.
- **Idempotence** :
  - un e-mail importé deux fois n'est enregistré qu'une seule fois (clé : identifiant Gmail du message) ;
  - une réponse ne part qu'une seule fois, même en cas de double clic ou de nouvelle tentative (clé d'idempotence).
- Mots de passe hachés, secrets dans des variables d'environnement, aucune donnée sensible dans les logs.

Partie 3 – Automatisation avec n8n

n8n self-hosted, en local. Découpez le traitement en **plusieurs workflows** :

1. **Synchronisation** (planifiée toutes les 10 minutes)
   - ne récupère que les e-mails arrivés depuis la dernière synchronisation réussie (curseur conservé côté backend) ;
   - gère la pagination de Gmail ;
   - extrait expéditeur, destinataires, objet, contenu texte, date, identifiant du message et identifiant du fil ;

    - envoie les e-mails au backend par lots.
2. **Classification par IA**, pour chaque nouvel e-mail, avec une sortie JSON **validée** :
    - catégorie parmi une liste fermée : *client*, *facture*, *rendez-vous*, *newsletter*, *spam probable*, *autre* ;
    - priorité de 1 à 3, résumé en une phrase, action attendue ;
    - si la réponse de l'IA n'est pas un JSON valide : une nouvelle tentative, puis catégorie *autre* et marquage « à vérifier ».
3. **Envoi des réponses**
    - déclenché par le backend, jamais directement par le site ;
    - la réponse part **dans le même fil Gmail** que le message d'origine ;
    - le statut est renvoyé au backend (envoyé ou échec).
4. **Gestion des erreurs** : workflow d'erreur (nœud `Error Trigger`) qui enregistre l'échec auprès du backend.

Respectez les limites du fournisseur d'IA : traitement par lots, pause entre les appels et reprise après une erreur de quota (HTTP 429).

Partie 4 – Qualité et exploitation

- Un `docker-compose.yml` qui lance l'ensemble (site, backend, base, n8n), avec un fichier `.env.example`
- Un **jeu de données de démonstration** : script injectant une trentaine d'e-mails fictifs, pour tester sans boîte Gmail
- Des **tests automatisés** couvrant au minimum :
    - l'import du même e-mail deux fois ;
    - l'envoi de la même réponse deux fois ;
    - le refus d'un webhook sans secret ou avec un secret erroné ;
    - le traitement d'une sortie IA valide et d'une sortie invalide.

Livrables attendus

Votre dépôt GitHub devra contenir :

- Le **site web**, le **backend** et ses **tests**.
- Les exports `.json` de **tous** les workflows n8n.
- Le `docker-compose.yml`, le `.env.example` et le script de données de démonstration.
- Un **rapport en Markdown** documentant :
    - les étapes d'installation (depuis un dépôt vierge) ;
    - l'architecture retenue (schéma) et le modèle de données ;
    - vos choix techniques et leurs raisons (base de données, découpage des workflows, synchronisation…) ;
    - les difficultés rencontrées et leurs solutions ;

- les résultats **mesurés** : durée de synchronisation de 30 e-mails, part des sorties IA valides du premier coup sur le jeu de démonstration ;
- les limites et les améliorations possibles.

⚠ Ajoutez `lilgar77` et `Holo795` comme collaborateurs sur votre dépôt.

Compétences évaluées

- Développement d'interfaces web claires et ergonomiques
- Conception d'une API et modélisation des données
- Automatisation de processus avec n8n et intégration d'un modèle d'IA
- Fiabilité : idempotence, synchronisation incrémentale, gestion des erreurs
- Sécurité des échanges entre le site, le backend et n8n
- Tests, qualité du code, documentation et organisation du projet

---

Recommandations

- Utilisez un compte Gmail de test.
- Commencez avec le jeu de données de démonstration, puis branchez Gmail.
- Pour l'IA : modèle gratuit via **Groq** ou équivalent.
- Ne stockez aucun identifiant en clair dans le code ni dans les exports de workflows.
- Faites des commits réguliers et explicites.

Documentation utile

- n8n – Documentation : `docs.n8n.io`
- API Gmail – messages et fils de discussion : `developers.google.com/gmail/api`

Critères d'évaluation

Critère	Points
Fonctionnalités complètes et opérationnelles (liste, filtres, fil, résumé, réponses)	30
Fiabilité : synchronisation incrémentale, absence de doublons et de double envoi, gestion des erreurs	20
Qualité du code et tests automatisés	15
Rapport clair et professionnel, avec résultats mesurés	15
Interface lisible, ergonomique et responsive	10
Sécurité (authentification, webhook, secrets)	10
Total	**100**

Bonus (non obligatoires)

- *Mise à jour de la liste en temps réel, sans rechargement (Server-Sent Events ou WebSocket)*
- *Signature HMAC du corps des webhooks en plus du secret partagé*
- *Masquage des numéros de téléphone et des IBAN avant l'envoi du contenu à l'IA*
- *Règle de réponse automatique sans validation pour une seule catégorie choisie, avec garde-fou documenté*